

Probabilistic Language Modeling

Sourav Karmakar

Outline

- 1 The Basics of Language Modelling
- 2 n -gram Language Modelling
- 3 Smoothing Techniques
- 4 Evaluating Language Models
- 5 Other Probabilistic Language Models

The Basics of Language Modelling

What is a Language Model?

A **Language Model (LM)** is a statistical or probabilistic model that computes a probability distribution over a sequence of words.

Given a sequence of words of length m , a language model assigns a probability $P(w_1, w_2, \dots, w_m)$ to the entire sequence. It captures the “likelihood” of a sequence occurring in a given language.

Example of LM Scoring

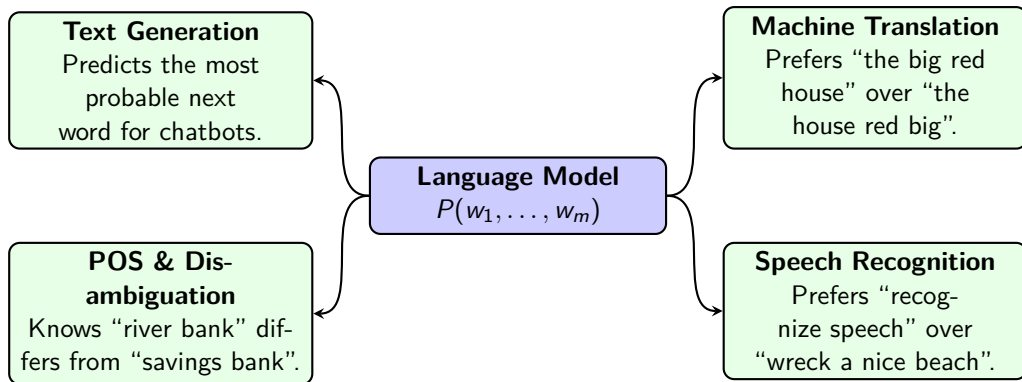
A well-trained English language model will assign probabilities such that:

$$P(\text{“the cat sat on the mat”}) \gg P(\text{“mat the on sat cat the”})$$

An LM that has learned the patterns of a language can **predict the next word** in a sequence or score how “natural” a given sentence sounds.

Why is Language Modeling Required?

Language models are the backbone of modern NLP. Their ability to quantify the plausibility of text enables machines to understand and generate human language.



n -gram Language Modelling

The Chain Rule and the Markov Assumption

By the strict rules of probability, calculating the likelihood of a sequence requires the **Chain Rule**:

$$P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i | w_1, \dots, w_{i-1})$$

Problem: Calculating probability based on the **entire** history is computationally expensive and suffers from severe data sparsity.

The Markov Assumption

We simplify the model by assuming the probability of the next word depends **only** on a fixed, finite number of previous words (specifically, $n - 1$).

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-n+1}, \dots, w_{i-1})$$

Unigram Language Modelling ($n = 1$)

The **unigram model** makes the strongest simplifying assumption: the probability of a word is completely independent of any other word in the text (“bag-of-words” assumption).

The probability of a word is its relative frequency in the training corpus:

$$P(w_i) = \frac{\text{count}(w_i)}{\sum_{j=1}^V \text{count}(w_j)} = \frac{\text{count}(w_i)}{N}$$

Where V is vocabulary size, and N is total tokens in the corpus.

The probability of an entire sentence $W = (w_1, w_2, \dots, w_k)$ is simply:

$$P(W) = \prod_{i=1}^k P(w_i)$$

Unigram Numerical Example

Corpus: “the cat sat on the mat. the dog sat.”

Total words (N) = 9. Vocabulary $V = \{\text{the, cat, sat, on, mat, dog}\}$. **Unigram Probabilities:**

- $P(\text{the}) = 3/9$
- $P(\text{cat}) = 1/9$
- $P(\text{sat}) = 2/9$
- $P(\text{dog}) = 1/9$

Calculate the probability of the sentence $W = \text{“the dog sat”}$:

$$\begin{aligned}P(\text{“the dog sat”}) &= P(\text{the}) \times P(\text{dog}) \times P(\text{sat}) \\&= \left(\frac{3}{9}\right) \times \left(\frac{1}{9}\right) \times \left(\frac{2}{9}\right) \\&= \frac{6}{729} \approx 0.0082\end{aligned}$$

Bigram Language Modelling ($n = 2$)

A **bigram model** takes one step up in complexity. It approximates the probability of a word w_i based only on the identity of the immediately preceding word w_{i-1} .

Estimated via counts from the corpus using Conditional Probability:

$$P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

The probability of a sentence W prepends a special start-of-sentence token $\langle s \rangle$:

$$P(W) = P(w_1|\langle s \rangle) \times \prod_{i=2}^k P(w_i|w_{i-1})$$

Bigram Numerical Example

Corpus: $\langle s \rangle$ the cat sat $\langle /s \rangle$ $\langle s \rangle$ the dog sat $\langle /s \rangle$

Relevant Counts:

- Unigrams: $\text{count}(\langle s \rangle) = 2$, $\text{count}(\text{the}) = 2$, $\text{count}(\text{cat}) = 1$.
- Bigrams: $\text{count}(\langle s \rangle, \text{the}) = 2$, $\text{count}(\text{the}, \text{cat}) = 1$, $\text{count}(\text{cat}, \text{sat}) = 1$.

Calculate the probability of $W = \text{"the cat sat"}$:

Bigram Numerical Example

Corpus: <s> the cat sat </s> <s> the dog sat </s>

Relevant Counts:

- Unigrams: $\text{count}(\text{<s>}) = 2$, $\text{count}(\text{the}) = 2$, $\text{count}(\text{cat}) = 1$.
- Bigrams: $\text{count}(\text{<s>, the}) = 2$, $\text{count}(\text{the, cat}) = 1$, $\text{count}(\text{cat, sat}) = 1$.

Calculate the probability of $W = \text{"the cat sat"}$:

$$P(\text{the}|\text{<s>}) = \frac{\text{count}(\text{<s>, the})}{\text{count}(\text{<s>})} = \frac{2}{2} = 1.0$$

Bigram Numerical Example

Corpus: $\langle s \rangle$ the cat sat $\langle /s \rangle$ $\langle s \rangle$ the dog sat $\langle /s \rangle$

Relevant Counts:

- Unigrams: $\text{count}(\langle s \rangle) = 2$, $\text{count}(\text{the}) = 2$, $\text{count}(\text{cat}) = 1$.
- Bigrams: $\text{count}(\langle s \rangle, \text{the}) = 2$, $\text{count}(\text{the}, \text{cat}) = 1$, $\text{count}(\text{cat}, \text{sat}) = 1$.

Calculate the probability of $W = \text{"the cat sat"}$:

$$P(\text{the}|\langle s \rangle) = \frac{\text{count}(\langle s \rangle, \text{the})}{\text{count}(\langle s \rangle)} = \frac{2}{2} = 1.0$$

$$P(\text{cat}|\text{the}) = \frac{\text{count}(\text{the}, \text{cat})}{\text{count}(\text{the})} = \frac{1}{2} = 0.5$$

Bigram Numerical Example

Corpus: $\langle s \rangle$ the cat sat $\langle /s \rangle$ $\langle s \rangle$ the dog sat $\langle /s \rangle$

Relevant Counts:

- Unigrams: $\text{count}(\langle s \rangle) = 2$, $\text{count}(\text{the}) = 2$, $\text{count}(\text{cat}) = 1$.
- Bigrams: $\text{count}(\langle s \rangle, \text{the}) = 2$, $\text{count}(\text{the}, \text{cat}) = 1$, $\text{count}(\text{cat}, \text{sat}) = 1$.

Calculate the probability of $W = \text{"the cat sat"}:$

$$P(\text{the}|\langle s \rangle) = \frac{\text{count}(\langle s \rangle, \text{the})}{\text{count}(\langle s \rangle)} = \frac{2}{2} = 1.0$$

$$P(\text{cat}|\text{the}) = \frac{\text{count}(\text{the}, \text{cat})}{\text{count}(\text{the})} = \frac{1}{2} = 0.5$$

$$P(\text{sat}|\text{cat}) = \frac{\text{count}(\text{cat}, \text{sat})}{\text{count}(\text{cat})} = \frac{1}{1} = 1.0$$

Bigram Numerical Example

Corpus: <s> the cat sat </s> <s> the dog sat </s>

Relevant Counts:

- Unigrams: $\text{count}(\text{<s>}) = 2$, $\text{count}(\text{the}) = 2$, $\text{count}(\text{cat}) = 1$.
- Bigrams: $\text{count}(\text{<s>, the}) = 2$, $\text{count}(\text{the, cat}) = 1$, $\text{count}(\text{cat, sat}) = 1$.

Calculate the probability of $W = \text{"the cat sat"}:$

$$P(\text{the}|\text{<s>}) = \frac{\text{count}(\text{<s>, the})}{\text{count}(\text{<s>})} = \frac{2}{2} = 1.0$$

$$P(\text{cat}|\text{the}) = \frac{\text{count}(\text{the, cat})}{\text{count}(\text{the})} = \frac{1}{2} = 0.5$$

$$P(\text{sat}|\text{cat}) = \frac{\text{count}(\text{cat, sat})}{\text{count}(\text{cat})} = \frac{1}{1} = 1.0$$

$$P(\text{"the cat sat"}) = 1.0 \times 0.5 \times 1.0 = \mathbf{0.5}$$

The Problem of Data Sparsity

Data sparsity is the most significant weakness of bigram (and higher-order) models. It occurs when a perfectly valid sequence of words does not appear in the training corpus.

Example: Using the exact same training corpus: $\langle s \rangle$ the cat sat $\langle /s \rangle$ $\langle s \rangle$ the dog sat $\langle /s \rangle$. Let's calculate $P(\text{"the cat sat on the mat"})$.

We must calculate $P(\text{on}|\text{sat})$:

$$P(\text{on}|\text{sat}) = \frac{\text{count}(\text{sat}, \text{on})}{\text{count}(\text{sat})}$$

The bigram "sat on" never occurs in our tiny corpus. Therefore, $\text{count}(\text{sat}, \text{on}) = 0$.

$$P(\text{on}|\text{sat}) = \frac{0}{2} = 0$$

The Zero-Probability Issue

Because the sentence probability is a product of its bigrams, $P(W) = \dots \times 0 \times \dots = \mathbf{0}$. The entire perfectly grammatical sentence is deemed impossible!

Trigrams and Higher-Order n -gram Models

We can generalize the Markov assumption: the probability of a word depends only on the previous $n - 1$ words.

$$P(w_i | w_{i-n+1}, \dots, w_{i-1}) = \frac{\text{count}(w_{i-n+1}, \dots, w_{i-1}, w_i)}{\text{count}(w_{i-n+1}, \dots, w_{i-1})}$$

The Trade-off of Increasing n :

- **Benefit (More Context):** A larger n captures longer-range word dependencies. The model learns patterns like “tickets are now on...” $\rightarrow$ “sale”.
- **Cost (Exponential Sparsity):** The number of possible 4-grams is vastly larger than bigrams. The vast majority will never be seen, worsening the zero-probability issue and requiring massive memory.

Smoothing Techniques

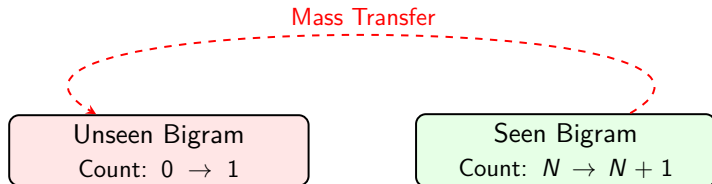
Laplace (Add-One) Smoothing

Smoothing addresses the zero-probability issue by redistributing some probability mass from seen n -grams to unseen n -grams.

Laplace Smoothing simply adds one to the count of every possible n -gram, including those never seen.

$$P_{\text{Laplace}}(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + \mathbf{1}}{\text{count}(w_{i-1}) + \mathbf{V}}$$

Note: We must add V (the vocabulary size) to the denominator to ensure all probabilities still sum to 1.



Laplace Smoothing Numerical Example

Let's fix the zero-probability issue for $P(\text{on}|\text{sat})$.

Corpus: `<s> the cat sat </s> <s> the dog sat </s>`

Variables:

- Vocabulary V : $\{\text{the, cat, sat, dog, on, mat}\} \rightarrow V = 6$.
- Original Counts: $\text{count}(\text{sat, on}) = 0$, $\text{count}(\text{sat}) = 2$.

Applying the Add-One Formula:

Laplace Smoothing Numerical Example

Let's fix the zero-probability issue for $P(\text{on}|\text{sat})$.

Corpus: `<s> the cat sat </s> <s> the dog sat </s>`

Variables:

- Vocabulary V : $\{\text{the, cat, sat, dog, on, mat}\} \rightarrow V = 6$.
- Original Counts: $\text{count}(\text{sat, on}) = 0$, $\text{count}(\text{sat}) = 2$.

Applying the Add-One Formula:

Laplace Smoothing Numerical Example

Let's fix the zero-probability issue for $P(\text{on}|\text{sat})$.

Corpus: <s> the cat sat </s> <s> the dog sat </s>

Variables:

- Vocabulary V : {the, cat, sat, dog, on, mat} $\rightarrow V = 6$.
- Original Counts: $\text{count}(\text{sat}, \text{on}) = 0$, $\text{count}(\text{sat}) = 2$.

Applying the Add-One Formula:

$$P_{\text{Laplace}}(\text{on}|\text{sat}) = \frac{\text{count}(\text{sat}, \text{on}) + 1}{\text{count}(\text{sat}) + V}$$

Laplace Smoothing Numerical Example

Let's fix the zero-probability issue for $P(\text{on}|\text{sat})$.

Corpus: <s> the cat sat </s> <s> the dog sat </s>

Variables:

- Vocabulary V : {the, cat, sat, dog, on, mat} $\rightarrow V = 6$.
- Original Counts: $\text{count}(\text{sat}, \text{on}) = 0$, $\text{count}(\text{sat}) = 2$.

Applying the Add-One Formula:

$$\begin{aligned}P_{\text{Laplace}}(\text{on}|\text{sat}) &= \frac{\text{count}(\text{sat}, \text{on}) + 1}{\text{count}(\text{sat}) + V} \\&= \frac{0 + 1}{2 + 6}\end{aligned}$$

Laplace Smoothing Numerical Example

Let's fix the zero-probability issue for $P(\text{on}|\text{sat})$.

Corpus: <s> the cat sat </s> <s> the dog sat </s>

Variables:

- Vocabulary V : {the, cat, sat, dog, on, mat} $\rightarrow V = 6$.
- Original Counts: $\text{count}(\text{sat}, \text{on}) = 0$, $\text{count}(\text{sat}) = 2$.

Applying the Add-One Formula:

$$\begin{aligned}P_{\text{Laplace}}(\text{on}|\text{sat}) &= \frac{\text{count}(\text{sat}, \text{on}) + 1}{\text{count}(\text{sat}) + V} \\&= \frac{0 + 1}{2 + 6} \\&= \frac{1}{8} = \mathbf{0.125}\end{aligned}$$

The probability is no longer zero!

Add- k (Lidstone) Smoothing

Problem: Add-One smoothing is often too blunt, stealing too much probability mass from actual seen events and giving it to absurd unseen events.

Solution: Add- k smoothing generalizes Laplace by adding a small fractional value k (where $0 < k < 1$) instead of 1.

$$P_{\text{Add-}k}(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + \mathbf{k}}{\text{count}(w_{i-1}) + \mathbf{kV}}$$

Example with $k = 0.1$:

Add- k (Lidstone) Smoothing

Problem: Add-One smoothing is often too blunt, stealing too much probability mass from actual seen events and giving it to absurd unseen events.

Solution: Add- k smoothing generalizes Laplace by adding a small fractional value k (where $0 < k < 1$) instead of 1.

$$P_{\text{Add-}k}(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + \mathbf{k}}{\text{count}(w_{i-1}) + \mathbf{kV}}$$

Example with $k = 0.1$:

Add- k (Lidstone) Smoothing

Problem: Add-One smoothing is often too blunt, stealing too much probability mass from actual seen events and giving it to absurd unseen events.

Solution: Add- k smoothing generalizes Laplace by adding a small fractional value k (where $0 < k < 1$) instead of 1.

$$P_{\text{Add-}k}(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + \mathbf{k}}{\text{count}(w_{i-1}) + \mathbf{kV}}$$

Example with $k = 0.1$:

$$P_{\text{Add-}0.1}(\text{on} | \text{sat}) = \frac{\text{count}(\text{sat}, \text{on}) + 0.1}{\text{count}(\text{sat}) + (0.1 \times V)}$$

Add- k (Lidstone) Smoothing

Problem: Add-One smoothing is often too blunt, stealing too much probability mass from actual seen events and giving it to absurd unseen events.

Solution: Add- k smoothing generalizes Laplace by adding a small fractional value k (where $0 < k < 1$) instead of 1.

$$P_{\text{Add-}k}(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + \mathbf{k}}{\text{count}(w_{i-1}) + \mathbf{kV}}$$

Example with $k = 0.1$:

$$\begin{aligned} P_{\text{Add-}0.1}(\text{on}|\text{sat}) &= \frac{\text{count}(\text{sat}, \text{on}) + 0.1}{\text{count}(\text{sat}) + (0.1 \times V)} \\ &= \frac{0 + 0.1}{2 + (0.1 \times 6)} = \frac{0.1}{2.6} \approx \mathbf{0.038} \end{aligned}$$

This provides a non-zero probability that is much more realistic.

Evaluating Language Models

Conceptual Understanding of Perplexity

How do we measure if our LM is actually good? The standard intrinsic metric is **Perplexity**.

What is Perplexity?

Perplexity is a measure of the model's "surprise" when encountering test data.

- **Low Perplexity:** Model is not surprised (accurately predicted the words).
- **High Perplexity:** Model is very surprised (data was unpredictable).

Intuition: The Weighted Average Branching Factor

A perplexity of 100 means that at every step, the model is as confused as if it had to choose between 100 different words with equal probability. Therefore, **lower is better**.

Mathematical Formulation of Perplexity

Perplexity is the inverse probability of the test set, normalized by the number of words N . For a test sentence $W = (w_1, \dots, w_N)$:

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}} = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

Expanding via the chain rule for a bigram model:

$$PP(W) = \left(\prod_{i=1}^N P(w_i | w_{i-1}) \right)^{-\frac{1}{N}}$$

To avoid floating-point underflow (multiplying tiny decimals), we compute it using the exponential of the cross-entropy (log probabilities):

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-1}) \right)$$

Numerical Example: Perplexity Setup

Let's calculate the perplexity using our Bigram model equipped with **Add-One smoothing**.

Model Details:

- Trained on: $\langle s \rangle$ the cat sat $\langle /s \rangle$ $\langle s \rangle$ the dog sat $\langle /s \rangle$

- Vocabulary Size (V): 7 (the, cat, sat, dog, on, mat, $\langle /s \rangle$)

Note: $\langle s \rangle$ is excluded — it only appears as a context/conditioning token, never as a word to be predicted. $\langle /s \rangle$ is included since the model must assign probability to predicting it.

Test Sentence (W): $\langle s \rangle$ the cat on the mat $\langle /s \rangle$

- Number of words (N): 6 (excluding start token, including end token).

We first calculate the smoothed bigram probabilities for this sequence:

$$\textcircled{1} P(\text{the}|\langle s \rangle) = \frac{2+1}{2+7} = \frac{3}{9} = \frac{1}{3}$$

$$\textcircled{2} P(\text{cat}|\text{the}) = \frac{1+1}{2+7} = \frac{2}{9}$$

Numerical Example: Perplexity Calculation

Continuing the smoothed probabilities:

- ③ $P(\text{on}|\text{cat}) = \frac{0+1}{1+7} = \frac{1}{8}$ (unseen bigram)
- ④ $P(\text{the}|\text{on}) = \frac{0+1}{0+7} = \frac{1}{7}$ (unseen bigram)
- ⑤ $P(\text{mat}|\text{the}) = \frac{0+1}{2+7} = \frac{1}{9}$
- ⑥ $P(</s>|\text{mat}) = \frac{0+1}{0+7} = \frac{1}{7}$ (unseen bigram)

Calculate Joint Probability $P(W)$:

$$P(W) = \frac{1}{3} \times \frac{2}{9} \times \frac{1}{8} \times \frac{1}{7} \times \frac{1}{9} \times \frac{1}{7} = \frac{2}{95256} \approx 0.0000210$$

Calculate Perplexity $PP(W)$:

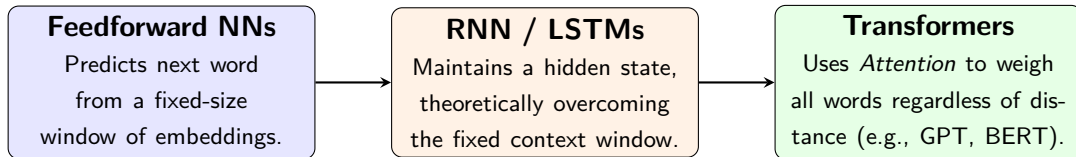
$$PP(W) = (0.0000210)^{-\frac{1}{6}} = \sqrt[6]{\frac{1}{0.0000210}} \approx \sqrt[6]{47628} \approx \mathbf{9.24}$$

Interpretation: The model is as confused as if it had to randomly choose between roughly 9 words at each step.

Other Probabilistic Language Models

Beyond n -grams: Modern LMs

While n -gram models laid the mathematical foundation, modern NLP has shifted to neural architectures that learn **continuous embeddings** rather than discrete counts.



These modern models drastically reduce the data sparsity problem and capture complex long-range semantic dependencies that n -grams cannot.

Thank You!

Any Questions?